

Interrupt Handling in RISC-V — Exam Notes

These notes are written for **understanding + exam preparation**, not just memorization.

1. What is an Interrupt?

An **interrupt** is a signal that temporarily stops the normal execution of a program so that the processor can handle an urgent event.

After handling the event, the processor returns to the program and continues execution from where it left off.

Definition (Exam Style)

An interrupt is a hardware or software event that causes the processor to suspend the current program execution, execute an Interrupt Service Routine (ISR), and then resume normal execution.

Why Interrupts are Needed

Without interrupts, the CPU would have to continuously check every device to see if it needs attention.

This process is called **polling**.

Interrupts are more efficient because devices notify the CPU only when service is required.

Advantages

- Better CPU utilization
 - Faster response to external events
 - Reduces unnecessary polling
 - Supports multitasking
-

2. Interrupt Handling in Intel vs RISC-V

Intel 8086/8088

Interrupt requests arrive through a dedicated hardware pin:

INTR

When a device needs service, it activates the INTR line.

RISC-V

RISC-V does not rely on a single interrupt pin like Intel.

Instead, interrupt requests are managed by:

- Platform-Level Interrupt Controller (PLIC)
- Core Local Interruptor (CLINT)

These controllers collect interrupt requests from many devices and inform the processor through special registers.

3. PLIC and CLINT

PLIC (Platform-Level Interrupt Controller)

Handles:

- External device interrupts
- Keyboard
- UART
- Network devices
- Other peripherals

Functions

- Collects interrupts from multiple devices
- Assigns priorities
- Delivers interrupt requests to CPU

CLINT (Core Local Interruptor)

Handles local interrupts such as:

- Timer interrupts
- Software interrupts

4. Important Registers Used in Interrupt Handling

These registers form the foundation of interrupt management in RISC-V.

4.1 mip Register

Full Form

Machine Interrupt Pending Register

Purpose

Stores information about pending interrupt requests.

When a device requests service, the corresponding bit inside mip becomes set.

Exam Point

mip indicates which interrupt sources are waiting to be serviced.

4.2 mstatus Register

Purpose

Stores processor status information.

Contains the important interrupt enable bit:

MIE

Machine Interrupt Enable

Values

Value	Meaning
MIE = 1	Interrupts enabled
MIE = 0	Interrupts disabled

Exam Point

mstatus stores the current processor state and interrupt enable status.

4.3 mie Register

Full Form

Machine Interrupt Enable Register

Purpose

Enables or disables specific interrupt sources.

Examples:

- Timer Interrupt
- Software Interrupt
- External Interrupt

Exam Point

mie acts as a mask register that controls which interrupts are allowed to generate processor interrupts.

4.4 mtvec Register

Full Form

Machine Trap Vector Register

Purpose

Stores the address of the Interrupt Service Routine (ISR).

When an interrupt occurs, the processor jumps to the address stored in mtvec.

Exam Point

mtvec contains the starting address of the interrupt handler.

4.5 mepc Register

Full Form

Machine Exception Program Counter

Purpose

Stores the address of the instruction that should execute after interrupt handling completes.

Why Needed?

After servicing the interrupt, the CPU must know where to return.

mepc provides this return address.

Exam Point

mepc stores the address of the interrupted program.

4.6 mcause Register

Purpose

Records the reason for the interrupt or exception.

Examples

- Timer interrupt
- External interrupt
- Software interrupt
- Exception

Exam Point

mcause identifies the source or cause of the interrupt.

5. Interrupt Service Routine (ISR)

Definition

An ISR is a special function executed when an interrupt occurs.

Responsibilities

- Identify interrupt source
- Perform required service
- Clear interrupt condition
- Return control to interrupted program

Exam Definition

An Interrupt Service Routine (ISR) is a software routine executed automatically in response to an interrupt.

6. Complete Interrupt Handling Sequence

Interrupt handling occurs in three phases.

Phase A: Hardware Response

These operations are performed automatically by hardware.

Step 1: Complete Current Instruction

The processor finishes the instruction currently being executed.

Reason

Prevents partial execution of instructions.

Step 2: Save Return Address

Address of the next instruction is saved into:

`mepc`

Step 3: Save Current State

Processor state is stored in:

`mstatus`

Step 4: Disable Further Interrupts

Global interrupt enable bit:

`MIE = 0`

This prevents nested interrupts.

Step 5: Jump to ISR

Program Counter (PC) loads the address stored in:

`mtvec`

The ISR begins execution.

Phase B: Software Response

Now the ISR executes.

Step 1: Save Registers

Unlike some CISC processors, RISC-V does not automatically save general-purpose registers.

The ISR must manually save registers to the stack.

Examples:

```
x1  
x2  
x3  
...  
x31
```

Step 2: Service Interrupt

The ISR performs the required task.

Examples:

- Reading UART data
 - Clearing timer interrupt
 - Handling keyboard input
-

Step 3: Restore Registers

All previously saved registers are restored from the stack.

Phase C: Return

The processor returns to normal program execution.

mret Instruction

ISR ends using:

```
mret
```

Functions of mret

1. Restore Interrupt State

Interrupts become enabled again.

```
MIE = 1
```

2. Restore Program Counter

PC is loaded with the address stored in:

```
mepc
```

3. Resume Program

Execution continues exactly where it was interrupted.

7. Why Does RISC-V Save Registers Manually?

RISC-V follows a Reduced Instruction Set Computer (RISC) philosophy.

To keep hardware simple:

- Hardware saves minimal information
- Software handles register saving

Benefits

- Simpler processor design
- Greater flexibility
- Faster hardware implementation

8. Comparison: Intel vs RISC-V

Feature	Intel x86	RISC-V
Interrupt Reception	INTR Pin	mip Register
Interrupt Controller	8259 PIC	PLIC / CLINT
ISR Address	Interrupt Vector	mtvec
Context Saving	Mostly Hardware	Mostly Software
Return Instruction	IRET	mret

9. Non-Maskable Interrupt (NMI)

Definition

An NMI is an interrupt that cannot be disabled or ignored.

Why is NMI Special?

Normal interrupts depend on:

`MIE bit`

NMI bypasses all interrupt-enable logic.

Even if interrupts are disabled:

`MIE = 0`

NMI will still be serviced.

Examples

- Memory corruption
 - Critical hardware failure
 - Serious system fault
 - Emergency shutdown conditions
-

Handling NMI

1. Separate Entry Address

Instead of `mtvec`, most systems jump to a special implementation-defined address.

2. Separate State Saving

NMI may occur while another ISR is already running.

Therefore many implementations use:

`mepc`

or dedicated backup registers.

3. Special Cause Code

`mcause` is updated with a special NMI code.

Most Important 5-Mark Question

Explain Interrupt Handling Sequence in RISC-V

Answer:

1. Interrupt request becomes pending in `mip`.
2. Processor completes current instruction.
3. Address of next instruction is saved in `mepc`.
4. Current status is saved in `mstatus`.

5. Interrupts are disabled by clearing MIE.
 6. Processor jumps to ISR address stored in mtvec.
 7. ISR saves required registers onto the stack.
 8. Interrupt service code executes.
 9. Registers are restored.
 10. ISR executes mret.
 11. PC is restored from mepc.
 12. Normal program execution resumes.
-

One-Line Revision Sheet (Night Before Exam)

mip → Pending interrupt?
mie → Which interrupts are enabled?
mstatus → Processor status + MIE bit
mtvec → Address of ISR
mepc → Return address
mcause → Why interrupt occurred?
mret → Return from interrupt

Hardware:
Finish instruction
Save mepc
Save mstatus
Disable interrupts
Jump to mtvec

Software:
Save registers
Service interrupt
Restore registers

Return:
mret
Restore PC from mepc
Resume execution

NMI:
Cannot be disabled
Used for critical failures
Bypasses normal interrupt logic

These notes are essentially what I'd personally revise from if I had a RISC-V interrupt handling question coming in an exam.